SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL2, BL3)*

Assessment Tool Weightage: 60%

QUESTIONS: 12 **Total Marks:60**

Annexure A

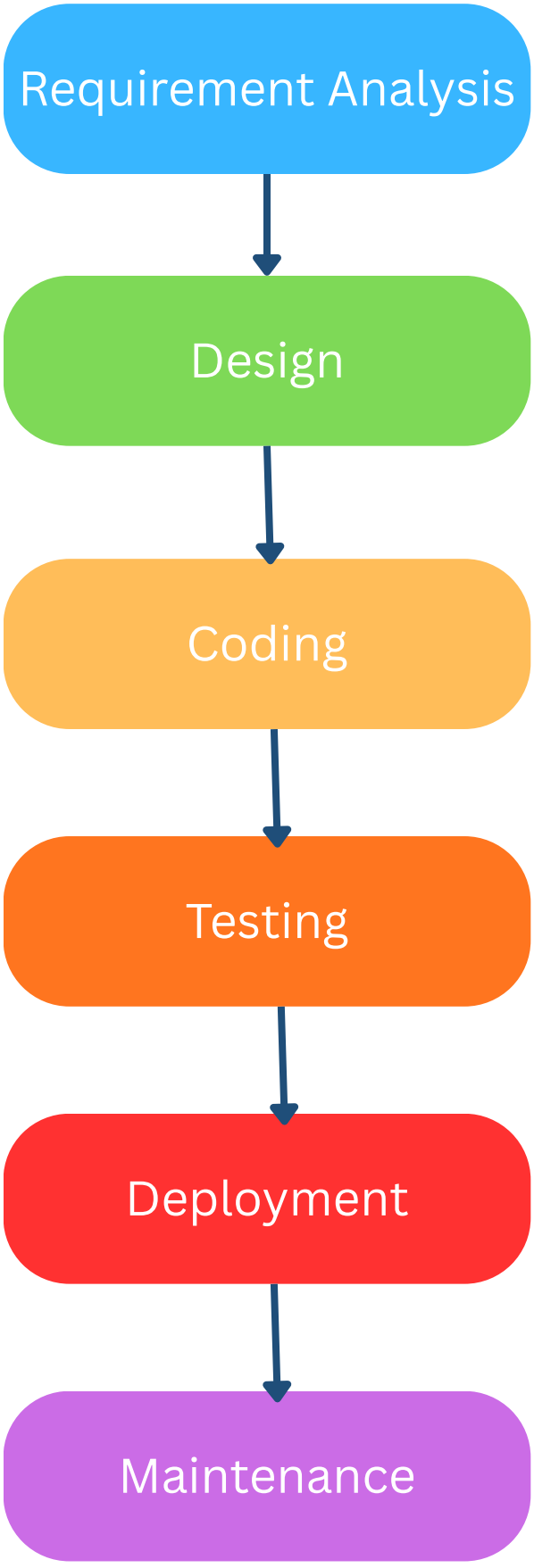
CONCEPT MAPPING

Code of Conduct:

I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Judy Allen J

Software Development Life Cycle



Explanation and Analysis

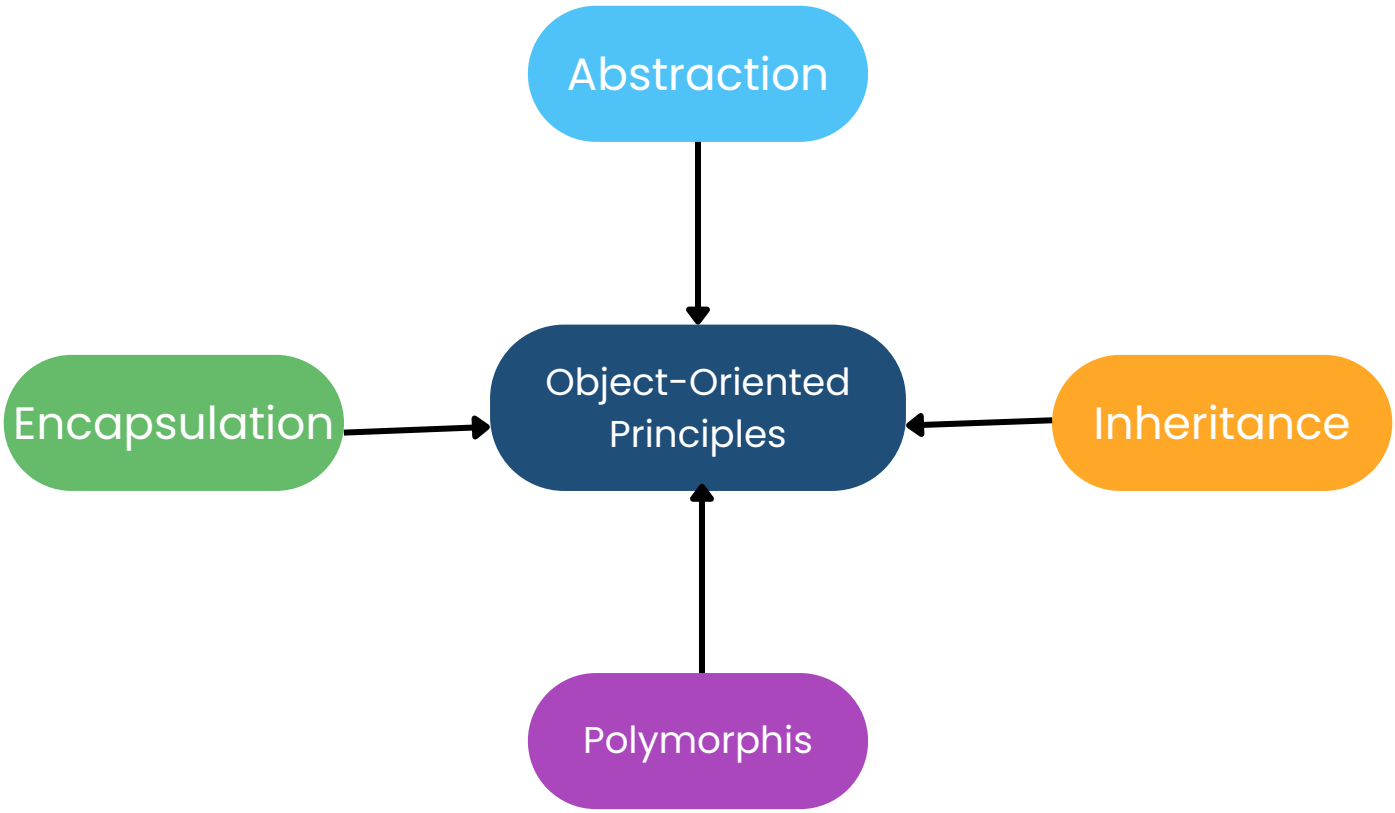
Explanation

- Requirement Analysis identifies user needs and system requirements.
- Design converts the requirements into a software architecture.
- Coding develops the software based on the design.
- Testing identifies and fixes software defects.
- Deployment releases the software to end users.
- Maintenance updates, enhances, and corrects the software after deployment.

Analysis

- Ensures a systematic software development process.
- Reduces project risks by following defined phases.
- Improves software quality and maintainability.
- Helps deliver reliable software on time.

Object Oriented Principles



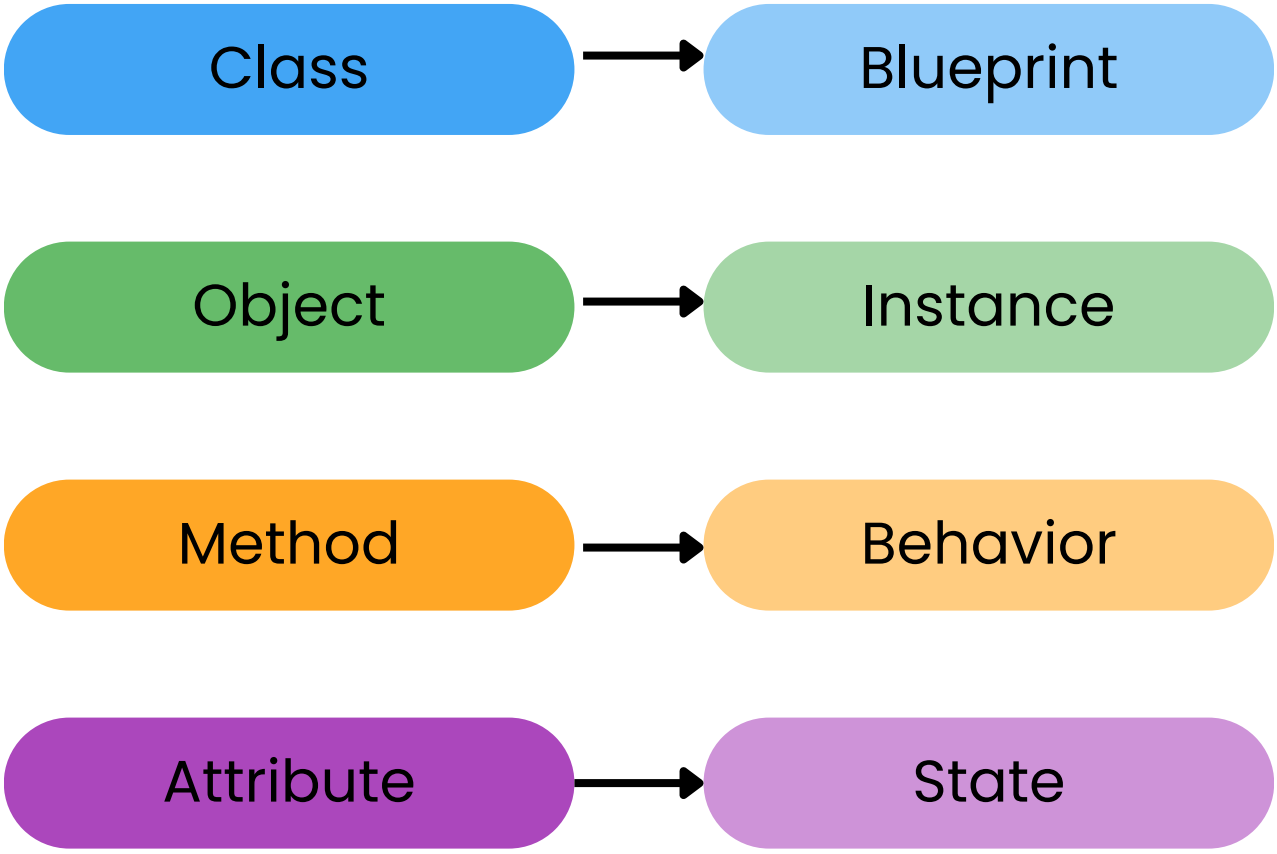
Explanation

- Abstraction hides unnecessary implementation details and exposes only essential features.
- Encapsulation protects data by combining data and methods within a class.
- Inheritance allows a class to inherit properties and methods from another class.
- Polymorphism enables one interface to perform different behaviors depending on the object.

Analysis

- Improves code reusability.
- Enhances data security.
- Increases software flexibility.
- Supports efficient object-oriented software design.

Relationship Between Class, Object, Method, and Attribute



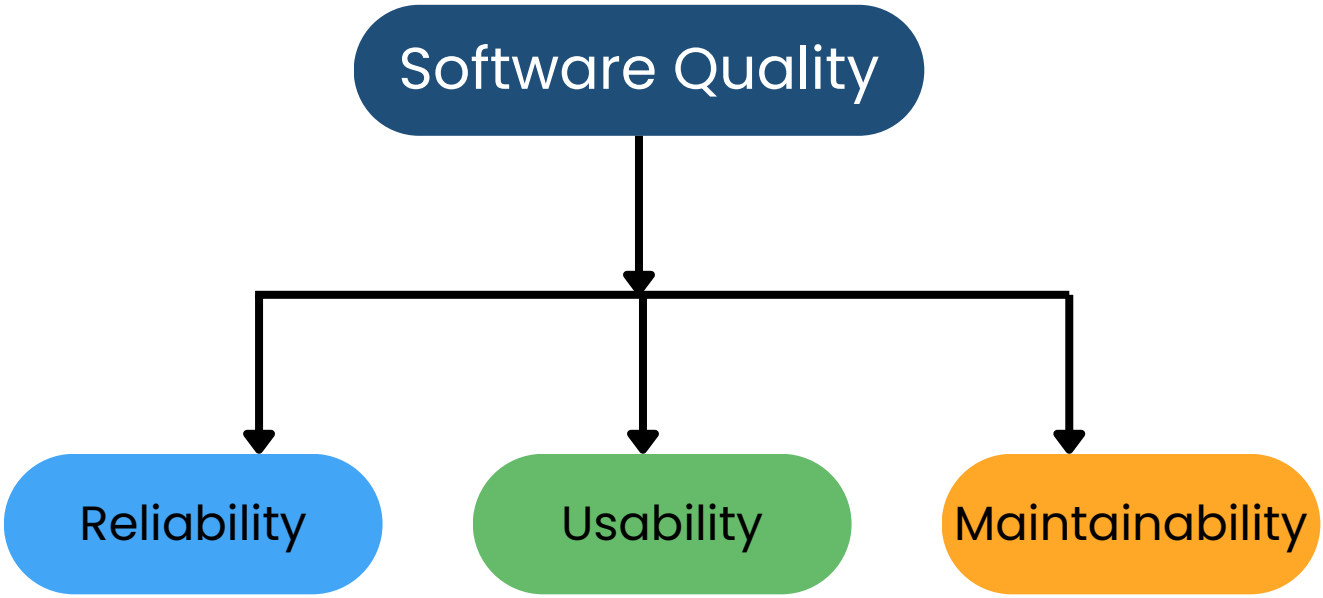
Explanation

- Methods define the behavior of objects.
- Attributes represent the state of objects.
- A class acts as a blueprint for creating objects.
- An object is an instance of a class.

Analysis

- Helps model real-world entities.
- Defines object characteristics and actions.
- Improves system organization.

Software Quality Attributes



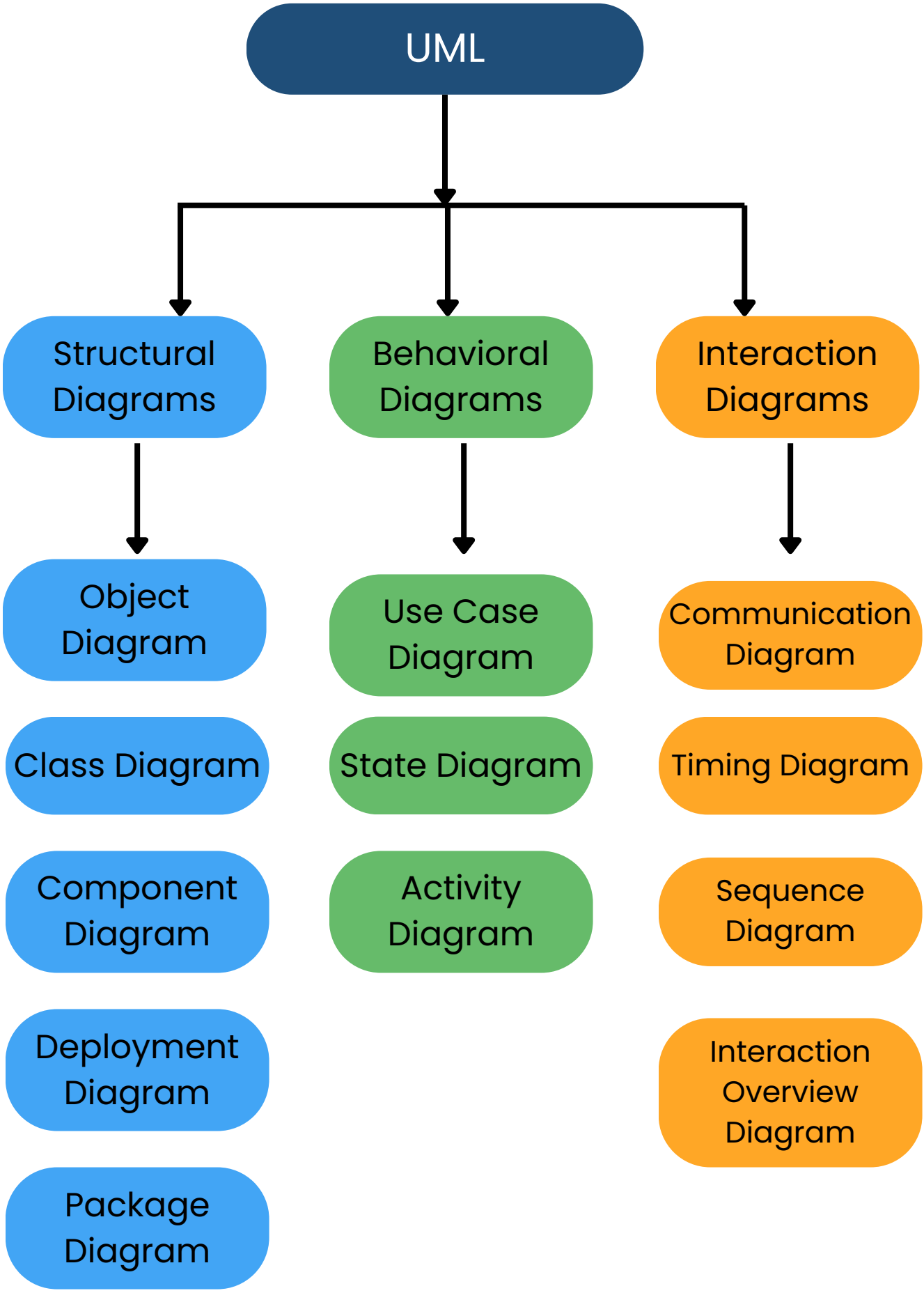
Explanation

- Reliability ensures the software performs consistently under expected conditions.
- Usability measures how easily users can learn and interact with the software.
- Maintainability makes the software easier to modify, update, and fix.

Analysis

- Improves user satisfaction.
- Enhances product acceptance.
- Contributes to software success.

Unified Modeling Language (UML) Diagram Classification



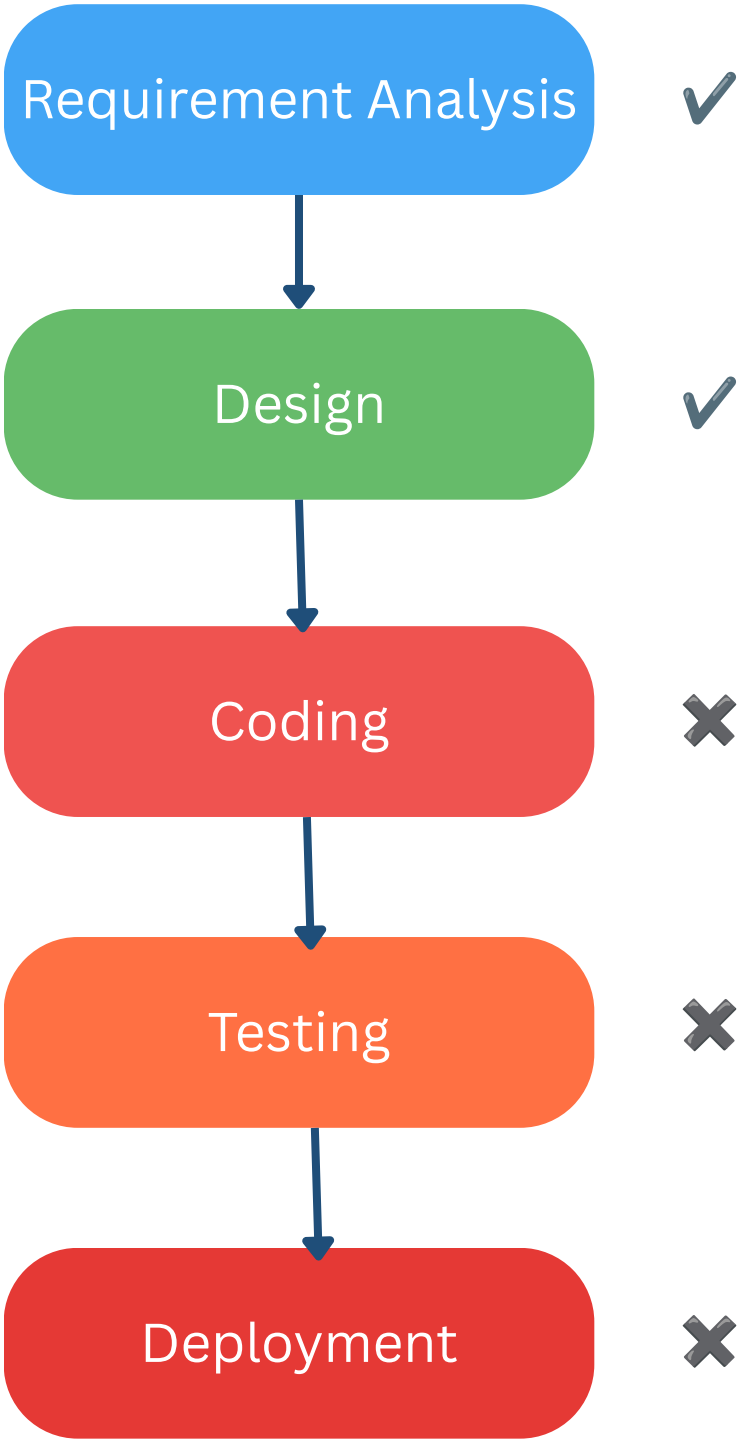
Explanation

- UML is a visual language used to design and model software systems.
- Structural Diagrams show the static structure of a system (e.g., Class, Object, Component).
- Behavioral Diagrams represent the system's behavior and workflow (e.g., Use Case, Activity, State).
- Interaction Diagrams show how objects communicate through messages (e.g., Sequence, Communication).

Analysis

- UML improves system visualization.
- It enhances communication among developers.
- It supports better software design and documentation.
- It simplifies software development and maintenance.

Software Development Failure During Coding Phase



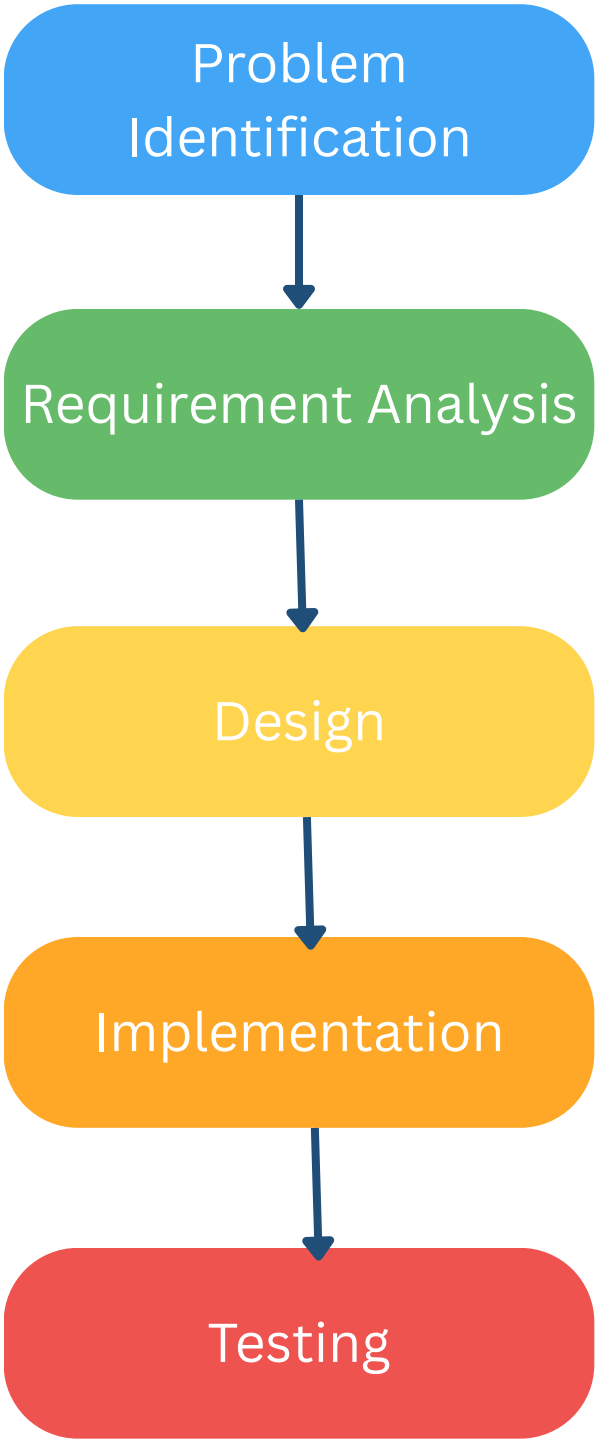
Explanation

- Requirements and design are completed successfully.
- Failure occurs during the coding phase due to programming errors.
- The coding failure affects the testing and deployment phases.

Analysis

- Coding errors reduce software quality.
- They delay project completion.
- Early code reviews and testing help prevent failures.

Software Development Process



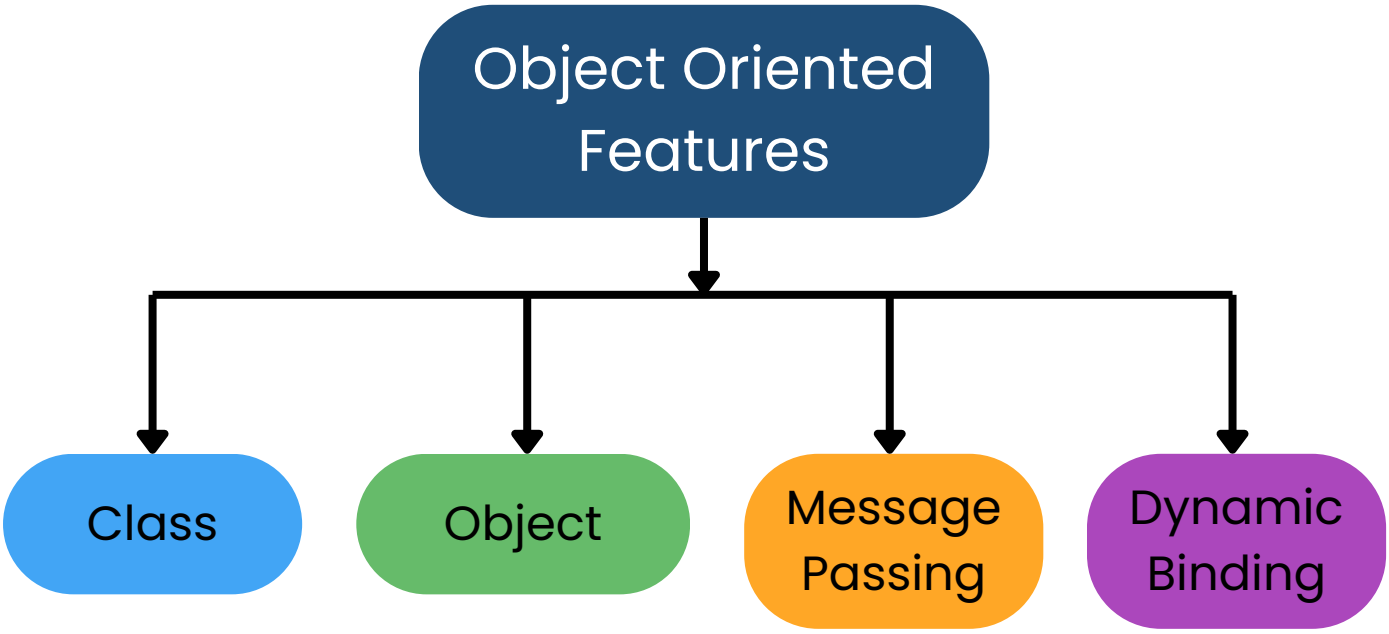
Explanation

- Requirement Analysis identifies user needs and system requirements.
- Testing verifies whether the implemented system meets the specified requirements.

Analysis

- Improves software quality.
- Reduces development errors.
- Ensures successful project completion.

Object-Oriented Features



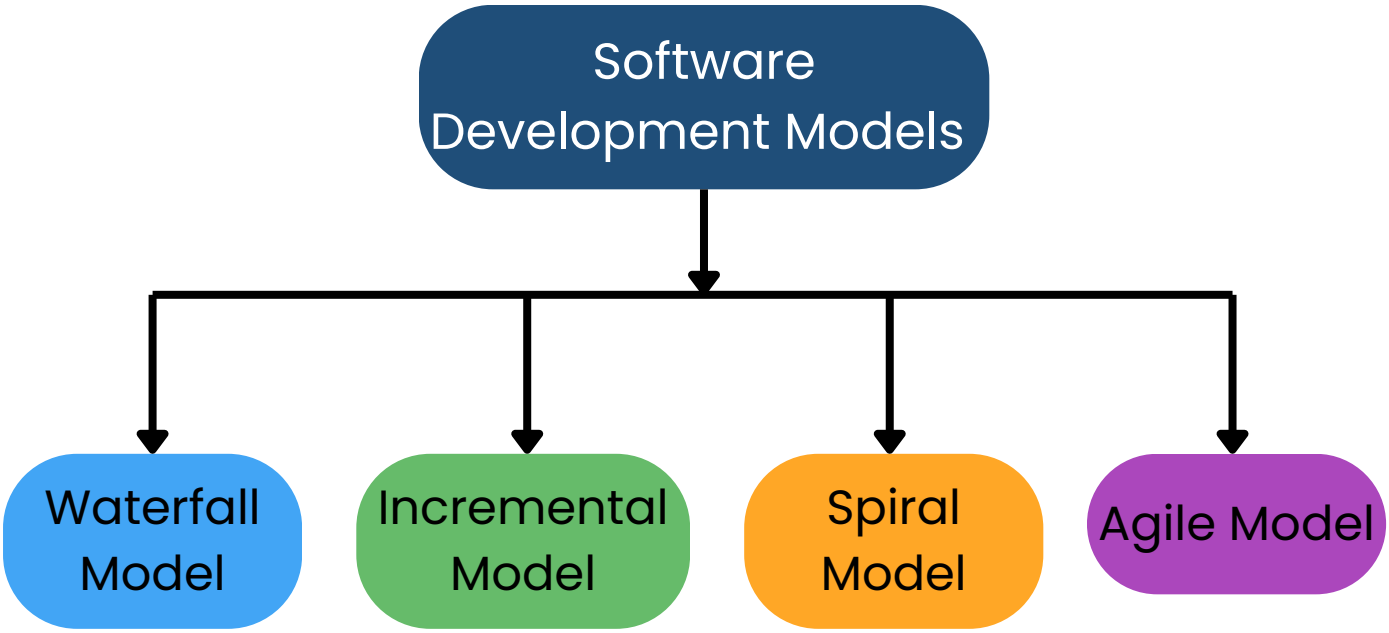
Explanation

- Objects are instances of classes.
- Dynamic binding resolves method calls during program execution.

Analysis

- Supports flexible software design.
- Improves code maintainability.
- Enables runtime polymorphism.

Software Development Models



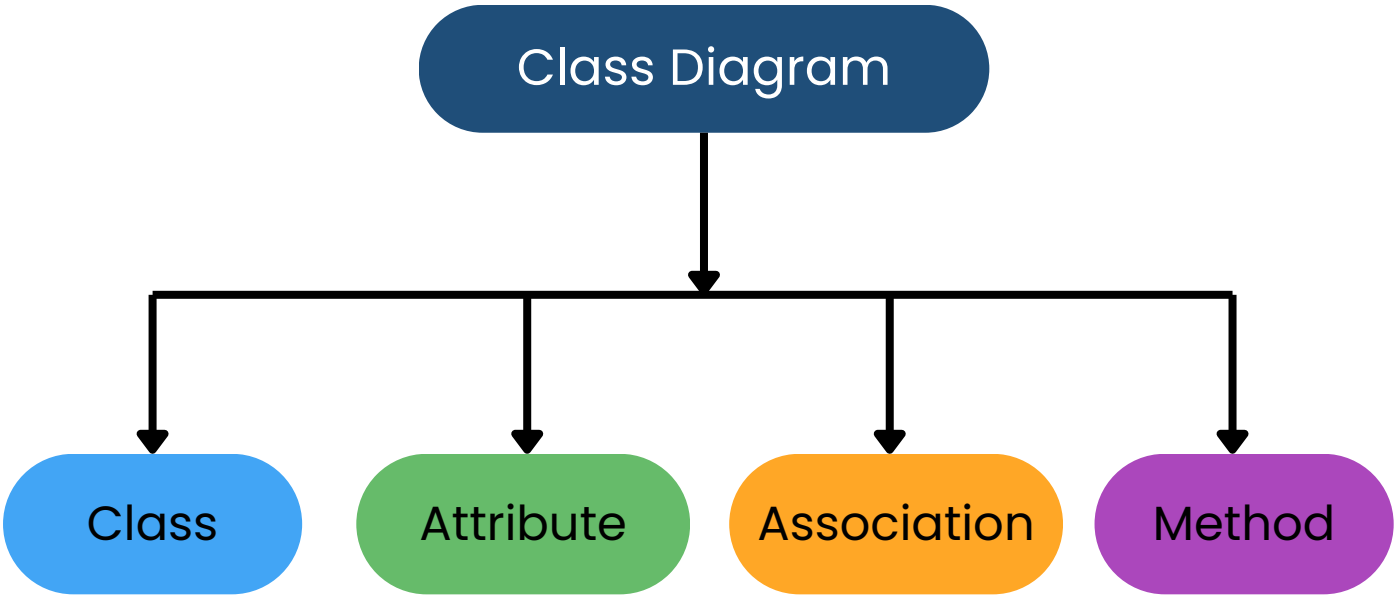
Explanation

- Incremental Model develops software in smaller stages.
- Agile Model supports iterative development with continuous customer feedback.

Analysis

- Enhances project adaptability.
- Reduces development risks.
- Improves customer satisfaction.

Class Diagram Components



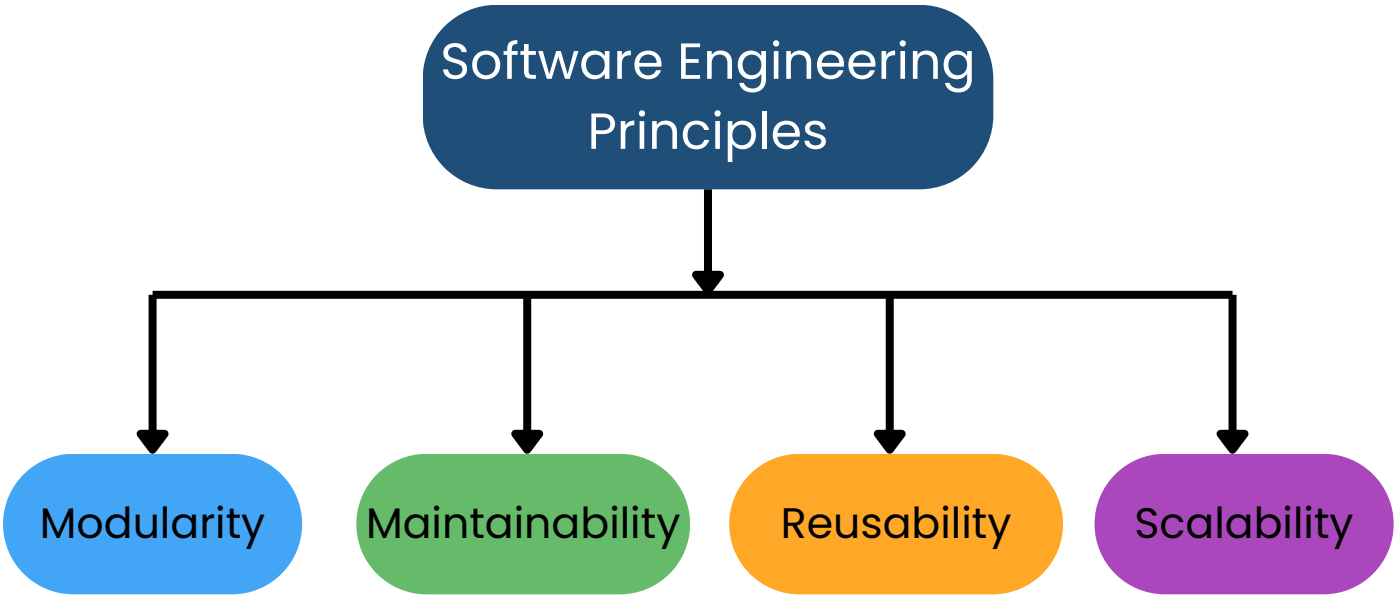
Explanation

- Attributes define the properties or data of a class.
- Methods define the functions or operations performed by a class.

Analysis

- Represents object structure clearly.
- Improves software documentation.
- Supports object-oriented design.

Software Engineering Principles



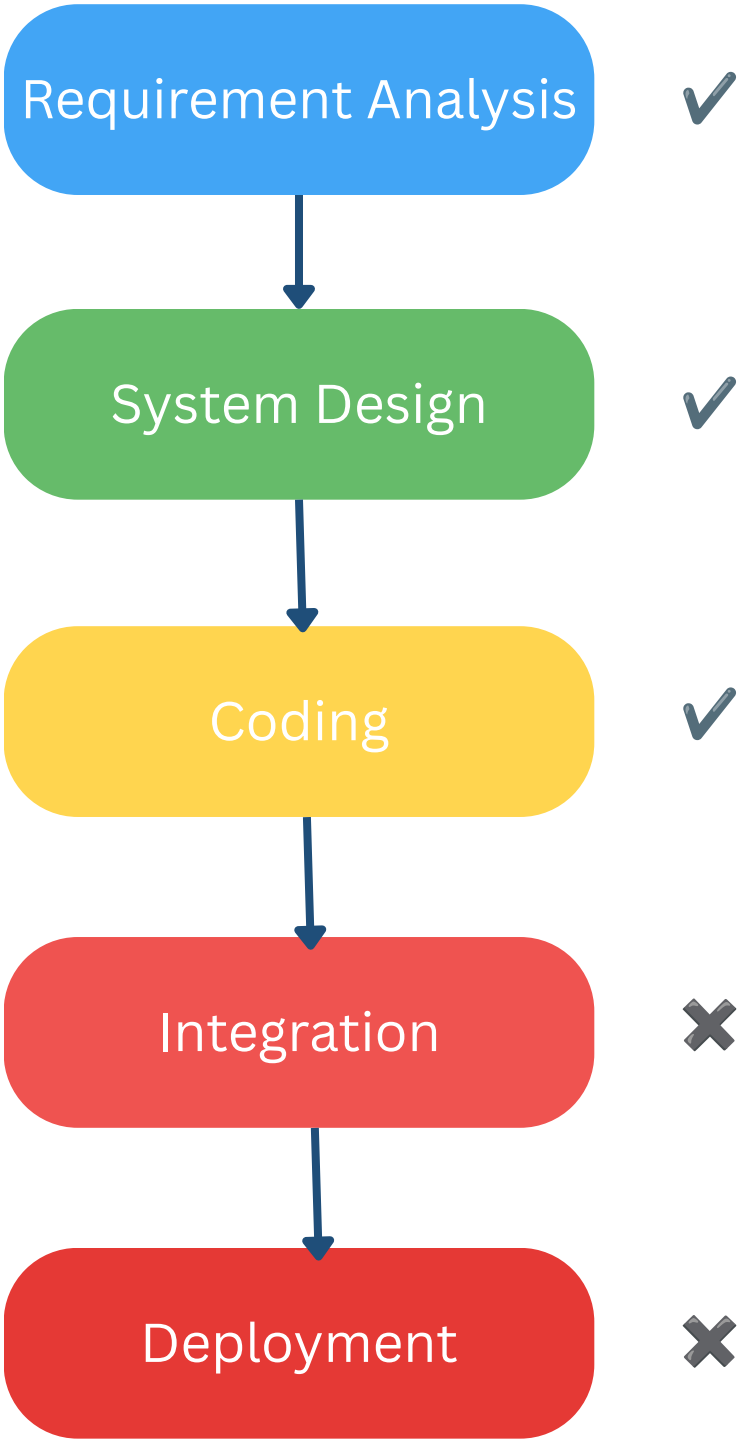
Explanation

- Maintainability makes software easier to update and fix.
- Scalability enables software to handle increasing users and workload efficiently.

Analysis

- Improves software performance.
- Reduces maintenance effort.
- Supports long-term software evolution.

Software Development Failure During Integration Phase



Explanation

- Failure occurs during the Integration phase.
- Individual modules fail to communicate correctly.

Analysis

- Causes functional inconsistencies.
- Delays software deployment.
- Increases debugging effort.